RAG ENGINE

Architecture & Technique Reference

Tool-Mapped Edition • MongoDB Atlas

Purpose
A structured reference for designing a production-oriented Retrieval-Augmented Generation (RAG) engine, covering ingestion, retrieval, reasoning, APIs, evaluation, and operational reliability.

Contents

- 1. Executive Summary
- 2. Architecture Overview & Tool Stack
- 3. Ingestion Layer
- 4. Services Layer
- 5. Core Layer
- 6. Retrieval Layer
- 7. Reasoning & Verification Layer
- 8. API Layer
- 9. Evaluation Techniques & Tools
- 10. Operational & Production Techniques
- 11. Three Credibility Invariants
- 12. Consolidated Tool Matrix
- 13. Key Design Trade-Offs
- Implementation Priorities

1. Executive Summary

A production-grade RAG engine is a multi-layered pipeline, not a single model call. Each layer has distinct techniques, failure modes, and tooling requirements.

The architecture follows a durable, event-driven pattern with five layers: Ingestion, Services, Core, Retrieval, and Reasoning/API. MongoDB Atlas serves as the central store for staged chunks, embedded knowledge, agent memory, and vector search.

The pipeline is source-agnostic at two seams: the data contract (for example, S3Ref) and the trigger core. New sources or file types should require only a new adapter or extractor class.

2. Architecture Overview & Tool Stack

Layer	Directory	Primary tools	Role
Ingestion	app/ingestion/	Docling / PyMuPDF / python-docx	Parse, chunk, and enrich raw documents
Services	app/services/	Voyage AI / OpenAI	Embedding generation and orchestration
Core	app/core/	MongoDB Atlas	Database client and index lifecycle
Retrieval	app/retrieval/	Atlas Vector Search + Atlas Search	Vector, keyword, and hybrid retrieval
Reasoning & API	app/reasoning/, app/api/	LLM via LangChain / Pydantic AI; FastAPI	Investigation, grounded generation, HTTP endpoints
Orchestration	Cross-cutting	Temporal	Durable, resumable, observable workflows

Architecture principle: keep ingestion and query-time reasoning modular, while using stable IDs and evidence records to preserve provenance end to end.

3. Ingestion Layer (app/ingestion/)

3.1 Multi-Format Parsing (parsers.py)

- Technique: extract text while preserving structural metadata such as PDF page numbers, Markdown heading hierarchy, and DOCX section boundaries. These become citation anchors downstream.
- Recommended tool: Docling for PDFs, Word, Markdown, audio, and hybrid chunking. Alternatives: PyMuPDF for PDF and python-docx for DOCX.

Function pattern: parse_pdf(file_bytes) -> (text, metadata_dict), where metadata includes source_file, page_number, and created_at.

3.2 Chunking Strategy (chunking.py)

A useful starting point is 300–800 tokens per chunk with 10–15% overlap. Select the algorithm according to document structure:

Chunking type	When to use	Technique / tool
Structural	Formatted documents with clear section boundaries	Split on headings, sections, and tables; custom logic or Docling hybrid chunker
Semantic	Unstructured narratives	Split where semantic similarity drops between sentences; LangChain SemanticChunker
Hierarchical parent-child	Need precise retrieval and broader context	Retrieve small child chunks; pass larger parent chunks to generation
Recursive character	General-purpose fallback	Separators: paragraphs, lines, sentences, spaces, then characters; LangChain

		RecursiveCharacterTextSplitter

- Deduplication: compute content_hash = sha256(chunk_text) so unchanged chunks are not re-ingested.
- Critical invariant: deterministic chunk IDs such as {doc_hash}:{page}:{chunk_index} are minted once and survive every pipeline stage.

3.3 Ingestion Orchestration (ingestion.py)

Pipeline: parse → chunk → deduplicate → embed → upsert.

- Use idempotent update_one with upsert=True, keyed on content_hash + source_id.
- Each chunk should carry text, embedding, source_id, content_hash, created_at, and metadata such as page, source_type, and owner.
- Use Temporal for durable workflows that can resume after failures.

4. Services Layer (app/services/)

4.1 Embedding Generation (embeddings.py)

- Pin the embedding model name and dimension in config.py. Mixing dimensions in one vector index breaks vector search.

Embedding model	Dimensions	Max tokens	Reported MTEB Avg	Reported cost	Reported latency
Voyage AI voyage-3	1024	32k	67.1	$0.06 / 1M tokens	~70 ms
OpenAI text-embedding-3-large	3072	Not specified here	64.6	$0.13 / 1M tokens	~80 ms
Cohere embed-v3 (English)	1024	Not specified here	64.5	$0.10 / 1M tokens	~60 ms

Decision guidance: consider Voyage AI or Cohere for retrieval quality in specialized domains; consider OpenAI text-embedding-3-small when low latency is the priority. Treat benchmark, price, and latency figures as reference estimates to revalidate against current provider documentation and your workload.

4.2 Idempotent Upsert Orchestration

Every chunk upsert uses update_one with upsert=True and a filter on content_hash + source_id. This avoids duplicate ingestion and keeps the indexed corpus clean.

5. Core Layer (app/core/)

5.1 Atlas Vector Search Index (database.py)

- Create a Vector Search index with the embedding field and declared filter fields such as source_type, created_at, and owner.
- Match numDimensions to the configured embedding model and use cosine similarity when appropriate.
- Make index management idempotent: check for existing indexes and support cutover via a configuration pointer.

5.2 Atlas Search (BM25) Index (database.py)

- Create a separate full-text search index for lexical/BM25 retrieval.
- Keyword search helps with exact identifiers, codes, names, and error IDs; vector search helps with semantic similarity.

6. Retrieval Layer (app/retrieval/)

6.1 Vector Search (vector_search.py)

- Use Approximate Nearest Neighbor search through the $vectorSearch aggregation stage.
- A starting heuristic is numCandidates around 10–20× the result limit. Tune it using a labeled evaluation set rather than assuming it is optimal.

6.2 Hybrid Search (hybrid_search.py)

Combine vector similarity with BM25 keyword search. Hybrid retrieval is useful when queries mix exact terminology and conceptual intent.

- Reciprocal Rank Fusion (RRF): score = Σ 1/(rank + k), commonly with k ≈ 60. It combines rank positions instead of incomparable raw scores.
- Where supported by the deployed Atlas version, use native $rankFusion or score-based $scoreFusion. Confirm feature availability and syntax against the target deployment.
- Fallback: merge ranked result sets in application code using manual RRF. LangChain's MongoDB Atlas hybrid retriever is another abstraction option.

6.3 Metadata Filtering

Tag chunks with attributes such as department, date, document type, and owner. Apply supported filters during vector retrieval where possible, rather than filtering only after retrieval.

6.4 Query Rewriting

Normalize and expand queries before embedding: correct spelling, resolve acronyms, and add missing context. Use lightweight rules first; introduce an LLM rewrite step when evaluation demonstrates value.

7. Reasoning & Verification Layer (app/reasoning/investigator.py)

7.1 Multi-Hop Retrieval via Query Decomposition

For complex questions, decompose the original query into sub-questions, retrieve evidence for each, merge the candidate pool, and rerank. An advanced Tree-of-Retrieval pattern branches into sub-queries and prunes low-quality branches using an evaluator.

7.2 Evidence Registry

Before generation, build a structured evidence registry such as [{chunk_id, text, source_id, page, score}]. This separates retrieval from generation and enables deterministic citation mapping.

7.3 Grounded Generation with Inline Citations

Prompt the model to attach an evidence marker to every factual claim, referencing the evidence registry. Inline citations improve traceability, but the prompt alone does not guarantee factual support.

7.4 Citation Verification (Post-Generation)

- Run a separate verification stage to check each answer claim against its cited evidence.
- Possible approaches named in the reference: VeriCite, ProofRAG, StillMe, Cite-Right, and Dokis. Validate maturity, availability, and suitability before adopting any project.

- Sentence-level alignment can use exact text checks, entailment/NLI models, or a combination. Mark unsupported claims explicitly rather than presenting them as established facts.

7.5 Unanswerable Query Handling

If evidence is insufficient, return a structured result such as {"answer": "I cannot find sufficient evidence.", "status": "unanswerable"}. Evaluate refusal behavior separately from answerable-query retrieval metrics.

An LLM-based IDK judge can help assess whether a response appropriately abstains, but should itself be evaluated.

7.6 Reranking

1. Semantic ranking: low-latency baseline; enable by default if it performs well in evaluation.
2. Cross-encoder reranking: test when additional latency is acceptable. Keep it only if measured relevance gains justify the cost; a 5-percentage-point nDCG gain is one possible project threshold.
3. LLM-as-reranker: reserve for a small final candidate set (for example, 3–5 items) when accuracy benefits justify the cost.

Voyage AI reranking and Cohere Rerank are provider options.

8. API Layer (app/api/)

Module / endpoint	Purpose	Example response / behavior
routes_ingest.py — POST /ingest	Accept file upload and trigger ingestion	{"status": "indexed", "chunks": count}
routes_investigate.py — POST /investigate	Run investigation and return grounded answer	Answer with citations and status: grounded or unanswerable
routes_eval.py — POST /eval/run	Run test cases and compute metrics	Evaluation report

Tooling: FastAPI for HTTP endpoints. LangChain offers MongoDB Atlas Vector Search integrations and LangSmith-based evaluation/tracing options.

9. Evaluation Techniques & Tools (app/api/routes_eval.py)

9.1 Retrieval Metrics

Metric	Definition	Question answered
Hit Rate@k	1 if at least one relevant item appears in top-k, averaged	Did the right source appear at all?
Recall@k	Fraction of all relevant items found in top-k	How much of the relevant set was covered?
Precision@k	Fraction of top-k items that are relevant	How noisy is the result set?
MRR	Mean reciprocal rank of the first relevant result	How high is the first relevant chunk?
nDCG@k	Normalized discounted cumulative gain	Are the most relevant results ranked near the top?

- MRR is less informative for multi-hop questions; use it when one key fact is sufficient.
- nDCG matters when context is truncated because it rewards placing highly relevant evidence near the top.

9.2 Generation Metrics

- Faithfulness: proportion of answer claims anchored in evidence.
- Answer relevance: how well the response addresses the question.
- Citation correctness: alignment between citations and assertions.
- Refusal quality: whether the system correctly abstains on unanswerable questions.
- Conflict handling rate: whether contradictory evidence is detected and handled.

9.3 Evaluation Framework Selection

Framework	Best for	Strength
RAGAS	Quick RAG assessment	RAG-focused metrics and synthetic test generation
DeepEval	CI/CD and broad testing	Metrics for RAG, agents, conversational AI, and safety
TruLens	Production monitoring	Feedback functions and ongoing evaluation
LangSmith	LangChain-integrated workflows	Tracing and evaluation workflow integration

Suggested stack: DeepEval in CI, RAGAS for RAG-focused metrics, and TruLens or Langfuse for production tracing and online evaluation. Select based on overlap, integration cost, and team needs.

9.4 Evaluation Dataset Construction

Build a golden set from production traffic (where privacy and consent permit), LLM-synthesized examples, and human-annotated cases.

Cover five categories: semantic paraphrases, exact identifiers, multi-hop questions, out-of-knowledge negative samples, and conflicting evidence.

Diagnostic rule of thumb: poor hit rate suggests retrieval issues; poor faithfulness suggests generation or evidence-use issues; faithful but incorrect answers may indicate source quality or ingestion problems.

9.5 Unanswerable Query Evaluation

Evaluate unanswerable queries separately from ordinary retrieval metrics. Measure whether the system correctly refuses when the corpus does not support an answer.

10. Operational & Production Techniques

10.1 Durable Orchestration

- Use Temporal when durable, resumable, observable workflows justify the added operational complexity.
- Event-driven ingestion can start from an object-created event, such as S3 ObjectCreated or a MinIO webhook. A message broker is not automatically required.

10.2 Idempotent Index Management

Make Atlas Vector Search and Atlas Search index creation repeat-safe. Check existing indexes and use a configuration pointer to support controlled index cutovers.

10.3 Multi-Stage Reranking

Start with a semantic baseline; add a cross-encoder only when evaluation demonstrates a meaningful nDCG improvement; use an LLM reranker only on a small final candidate set.

10.4 Dynamic Knowledge Source Routing

Route queries to an appropriate index when distinct knowledge domains require different retrieval settings. Start with one index when a single corpus and configuration are sufficient.

10.5 Embedding Model Discipline

Pin model name and dimensions in configuration. Start with a general-purpose embedding model and adopt domain-specific models such as PubMedBERT or BioBERT only when evaluation on the actual workload shows measurable improvement.

11. Three Credibility Invariants

4. Deterministic chunk IDs: mint once (for example, {doc_hash}:{page}:{chunk_index}) and preserve across all pipeline stages.
5. Idempotent ingestion: use content hashes and stable source identifiers to prevent unchanged content from being duplicated.
6. Post-generation verification: independently check claims against cited source text; never trust a model's citation merely because it produced one.

12. Consolidated Tool Matrix

Capability	Primary tool	Alternative / fallback
Document parsing	Docling	PyMuPDF, python-docx
Chunking	Docling hybrid chunker / LangChain SemanticChunker	Custom recursive character splitter
Embeddings	Voyage AI voyage-3	OpenAI text-embedding-3-large, Cohere embed-v3
Vector storage/search	MongoDB Atlas Vector Search ($vectorSearch)	—
Keyword search (BM25)	MongoDB Atlas Search ($search)	—
Hybrid fusion	Atlas $rankFusion / $scoreFusion where supported	Manual RRF
Reranking	Voyage AI reranker	Cohere Rerank
Orchestration	Temporal	Simpler application/background tasks if durability needs are modest
Agent framework	Pydantic AI	LangChain / LangGraph
API layer	FastAPI	—
Evaluation	RAGAS + DeepEval + TruLens	LangSmith
Citation verification	Research/project options: VeriCite / ProofRAG / StillMe	Custom NLI-based verifier
Unanswerable detection	IDK-style judge	Rules plus evaluated LLM abstention classifier
Database client	MongoDB Atlas	—

13. Key Design Trade-Offs

Decision	Guidance
Hybrid vs. pure vector	Use hybrid when exact terminology and conceptual intent coexist. Pure vector can suffice for purely conceptual corpora, but validate with tests.
Reranking depth	Use semantic ranking as baseline; add cross-encoder if measured nDCG gain justifies latency; reserve LLM reranking for a few candidates.
Single vs. multi-index	Start with one index; add routing when query types need materially different retrieval settings.
Structural vs. semantic chunking	Structural for well-formatted documents; semantic for narratives; parent-child when precise retrieval and broad context are both needed.
General vs. domain-specific embeddings	Start general-purpose; adopt specialized embeddings only when evaluation supports the change.
Native vs. manual RRF	Use native $rankFusion when supported by the deployed Atlas environment; otherwise fuse ranked lists in application code.

Implementation Priorities

The highest-leverage risks identified in this reference are fragmented chunk boundaries and bypassed citation verification. Poor chunk boundaries can make evidence hard to retrieve, and unverified citations undermine trust.

7. Establish deterministic IDs, content hashes, and source metadata before adding advanced reasoning.
8. Build a small labeled evaluation set early and use it to compare chunking, retrieval, and reranking choices.
9. Implement hybrid retrieval only after the vector and lexical baselines can be measured independently.
10. Make the evidence registry and citation verification explicit stages in the request flow.
11. Revalidate fast-changing claims—provider pricing, benchmark scores, API syntax, feature availability, and project maturity—against current primary documentation before implementation.